

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43
NAME: E.HEMANTH REDDY
REG.NO.: 192372098

COURSE OUTCOME COVERED

CO-3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Technical Presentation
Weightage: 20%

QUESTIONS:

Total Marks: 20

1. Servlet Architecture & Workflow

Question:

Present the **architecture of Java Servlets** and explain the complete **request–response cycle** with a neat diagram.

Focus Areas:

- Client–Server interaction
- Role of Web Container
- Request and Response objects

2. Servlet Life Cycle Analysis

Question:

Explain the **Servlet Life Cycle** with a sequence diagram and analyze the role of `init()`, `service()`, and `destroy()` methods.

Focus Areas:

- Life cycle phases
- Execution flow
- Performance implications

3. GET vs POST Method Evaluation

Question:

Compare **GET and POST methods** used in form submission. Present scenarios where each method is appropriate.

Focus Areas:

- Data transmission
 - Security aspects
 - URL limitations
-

4. Session Tracking Techniques**Question:**

Present different **session tracking mechanisms** (HttpSession, Cookies, URL rewriting). Evaluate their advantages and limitations.

Focus Areas:

- Session persistence
 - Security concerns
 - Use-case comparison
-

5. Cookies vs Sessions – Critical Analysis**Question:**

Analyze and compare **Cookies and Sessions** in web applications. Present real-time use cases.

Focus Areas:

- Storage location
 - Security risks
 - Performance considerations
-

Code of Conduct:

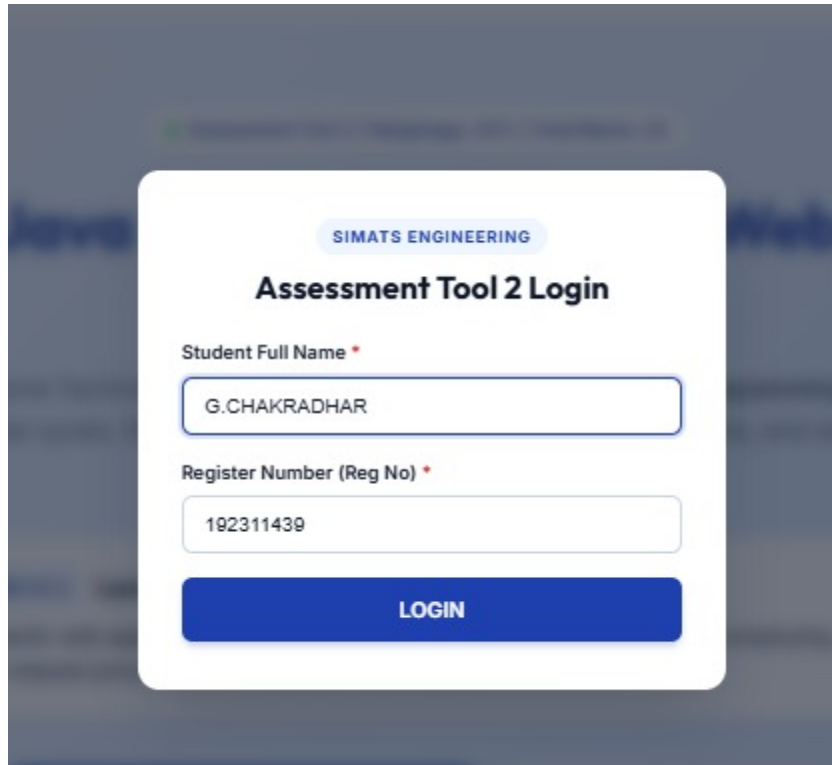
*I **E.HEMANTH REDDY(192372098)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: E.HEMANTH REDDY

RUBRICS FOR CALCULATION:

Technical Presentation					
Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

ANSWERS



The image shows a login form for 'SIMATS ENGINEERING'. The form is titled 'Assessment Tool 2 Login'. It contains two input fields: 'Student Full Name' with the value 'G.CHAKRADHAR' and 'Register Number (Reg No)' with the value '192311439'. A blue 'LOGIN' button is at the bottom.

SIMATS ENGINEERING

Assessment Tool 2 Login

Student Full Name *

G.CHAKRADHAR

Register Number (Reg No) *

192311439

LOGIN

Java Servlets

CSA43 - CO3

Overview

Page 1: Architecture

Page 2: Life Cycle

Page 3: GET vs POST

Page 4: Sessions

Page 5: Cookies vs Sessions

G

G.CHAKRADHAR 192311439

Change

Java Servlets & Dynamic Web Architecture

Comprehensive Technical Presentation solutions for **CSA43 - Internet Programming**. Covering request-response cycles, lifecycle execution flows, HTTP transmission methods, and state persistence techniques.

COURSE OUTCOME CO-3

Learning Outcome Statement

Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. *(Level 3)*

Explore Page 1: Servlet Architecture →

View Rubrics & Integrity

Student Code of Conduct

"I **G.CHAKRADHAR (192311439)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action."

Signature of Student:

Signed by G.CHAKRADHAR (192311439)

Question-1

Servlet Architecture & Workflow

Question: Present the architecture of Java Servlets and explain the complete request-response cycle with a neat diagram.

Focus Areas:

Client-Server Interaction

Role of Web Container

Request & Response Objects

STEP 1

Main Components Breakdown

Client / Browser

The user interface or web application that initiates HTTP requests (e.g., GET or POST) to access dynamic web resources.

Web Server

Accepts raw HTTP requests from network sockets and forwards them to the underlying Web Container environment.

Web Container

The servlet engine (e.g., Apache Tomcat, Jetty) managing servlet lifecycles, memory, security, and request dispatching.

STEP 1 Main Components Breakdown



Client / Browser

The user interface or web application that initiates HTTP requests (e.g., GET or POST) to access dynamic web resources.



Web Server

Accepts raw HTTP requests from network sockets and forwards them to the underlying Web Container environment.



Web Container

The servlet engine (e.g., Apache Tomcat, Jetty) managing servlet lifecycles, memory, security, and request dispatching.



Java Servlet

A server-side Java component executing business logic, database queries, and generating dynamic content.



HttpServletRequest

Encapsulates client parameters, request headers, cookies, HTTP methods, and input streams for processing.



HttpServletResponse

Used by the servlet to configure status codes, response headers, content types, and output HTML/JSON streams.



Java Servlets

CSA43 - C03

Overview

Page 1:
Architecture

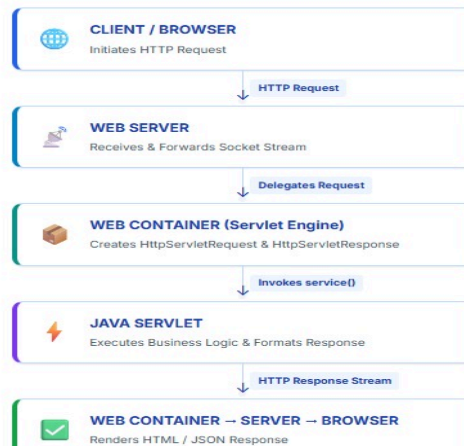
Page 2: Life
Cycle

Page 3: GET vs
POST

Page 4:
Sessions

Page 5: Cookies vs
Sessions

STEP 2 Architecture & Workflow Process Diagram



STEP 3 Complete Request-Response Cycle Execution Steps

1. **User Action:** User enters a URL or submits an HTML form in the client browser.
2. **Request Packaging:** The browser constructs an HTTP request packet (e.g., GET or POST).
3. **Web Server Reception:** The request reaches the web server port (e.g., 8080).
4. **Servlet Mapping:** The web container maps the incoming URL pattern to a specific servlet class.
5. **Object Instantiation:** The container creates `HttpServletRequest` and `HttpServletResponse` instances.
6. **Service Invocation:** The container spawns/allocates a thread and calls the servlet's `service()` method.
7. **Handler Dispatch:** `service()` inspects the request method and dispatches to `doGet()`, `doPost()`, etc.
8. **Logic Execution:** Servlet performs business logic, database queries, or data transformations.
9. **Response Generation:** Servlet writes content into `HttpServletResponse.getWriter()` output stream.
10. **Response Delivery:** The container flushes data through the server to the browser for DOM rendering.

STEP 4 Implementation Example (Java Servlet)

HelloServlet.java

```
import jakarta.servlet.http.*;
import jakarta.servlet.*;
import java.io.*;
```

STEP 4 Implementation Example (Java Servlet)

HelloServlet.java

Copy Code

```
import jakarta.servlet.http.*;
import jakarta.servlet.*;
import java.io.*;

public class HelloServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        // 1. Set response content type
        response.setContentType("text/html");

        // 2. Obtain writer stream
        PrintWriter out = response.getWriter();

        // 3. Write HTML output
        out.println("<h1>Hello from Servlet</h1>");
    }
}
```

Question-2

Servlet Life Cycle Analysis

Question: Explain the Servlet Life Cycle with a sequence diagram and analyze the role of `init()`, `service()`, and `destroy()` methods.

Focus Areas:

Life Cycle Phases

Execution Flow

Performance Implications

STEP 1 The 4 Core Life Cycle Phases

1

Loading & Instantiation

The container loads the servlet class file via `ClassLoader` and instantiates the servlet object.

2

Initialization

The container invokes `init(ServletConfig)` **exactly once** to initialize resources before handling requests.

3

Request Processing

The container calls `service()` for **every incoming request**, dispatching to specific HTTP methods.

4

Destruction

The container invokes `destroy()` **once** prior to servlet garbage collection or container shutdown.

STEP 2 Sequence Diagram Visualization

1

Client HTTP Request

Client browser issues an HTTP request targeted at a mapped servlet URL pattern.

2

Web Container Instantiation & Loading

Container checks if servlet class instance exists. If absent, loads class and creates instance.

3

`init()` Execution (Executed Once)

Container calls `init()` to set up configurations, database connections, or cache objects.

4

`service(req, res)` Execution (Executed Per Request)

Container spawns worker thread and dispatches request to `doGet()` / `doPost()`.

5

`destroy()` Execution (Executed Once)

Container unloads servlet instance, invoking `destroy()` to release open streams, sockets, and DB connections.

STEP 3 Lifecycle Method Analysis

Method	Invocation Frequency	Primary Purpose	Performance Note
init()	Executed ONCE	Initialize global configuration resources, database connections, or file handles.	Ideal location for heavy one-time startup tasks to minimize request overhead.
service()	Executed PER REQUEST	Processes incoming requests and responses; dispatches to doGet() / doPost().	Must be lightweight, optimized, and thread-safe due to concurrent multi-client access.
destroy()	Executed ONCE	Releases active resources, closes database connections, flushes files before removal.	Ensures clean resource deallocation and prevents server memory leaks.

STEP 4 Performance & Concurrency Implications

⚡ Critical Performance Guidelines

- **One-Time Initialization:** Heavy setup actions (e.g., database connection pool initialization, reading config files) should always occur in `init()`, never inside `service()` or `doGet()`.
- **Thread-Safety Caution:** By default, a web container uses a single servlet instance to service concurrent client requests via multi-threading. Avoid declaring mutable instance variables in servlets to prevent race conditions.
- **Graceful Shutdown:** Always release sockets, IO channels, and external database pools in `destroy()` to avoid memory leaks during redeployment.

Question-3

PAGE 3 OF 5 QUESTION 3

GET vs POST Method Evaluation

Question: Compare GET and POST methods used in form submission. Present scenarios where each method is appropriate.

Focus Areas: Data Transmission Security Aspects URL Limitations

GET Method Mechanics

- Appends form parameters directly to the URL query string (e.g., `/search?keyword=java`).
- Designed primarily for **safe, idempotent data retrieval**.
- URL can be easily bookmarked, cached, and shared.
- **Security Risk:** Sensitive credentials (passwords, SSNs) are exposed in browser history, proxy server logs, and referrer headers.

POST Method Mechanics

- Encapsulates parameter data inside the HTTP request body.
- Designed for **state-changing submissions** (creating orders, user registration, updates).
- Data is hidden from the browser address bar and not cached.
- **Security Note:** POST does not encrypt data by default; HTTPS is strictly mandatory for confidentiality.

STEP 3

Comprehensive GET vs POST Comparison Matrix

Feature	GET Method	POST Method
Data Transmission	URL Query String (e.g., ?key=val)	HTTP Request Message Body
Visibility	Visible in Address Bar, Logs, and History	Hidden from Address Bar
Payload Limit	Restricted by browser URL length (~2,048 chars)	No artificial limit (handles large payloads/files)
Bookmarking & Caching	Can be cached and bookmarked easily	Cannot be bookmarked; browser warns on refresh
Idempotency & Safety	Idempotent (multiple requests yield same state)	Non-idempotent (creates/modifies server data)
Security Consideration	Insecure for sensitive data (visible in URL)	More secure for submission; requires HTTPS
Typical Purpose	Retrieve / Search / Filter Public Data	Submit Forms / Login / Payments / File Uploads
Real-Time Example	Google Search Query, E-commerce Filtering	User Registration, Checkout Payment Processing

STEP 4

Recommended Usage Scenarios

When to Use GET

When to Use POST

STEP 4

Recommended Usage Scenarios

✓ When to Use GET

- Search engines & product catalog filtering.
- Pagination links (e.g., page=2).
- Viewing public articles, blog posts, or static resources.
- Operations that do not alter database state.

✓ When to Use POST

- User login & authentication forms (passwords).
- Registration forms & profile updates.
- Financial transaction & e-commerce order submissions.
- Uploading files, images, or large text documents.

STEP 5

HTML Form Code Syntax Examples

GET Form Example

POST Form Example

Question-4

PAGE 4 OF 5

QUESTION 4

Session Tracking Techniques

Question: Present different session tracking mechanisms (HttpSession, Cookies, URL rewriting). Evaluate their advantages and limitations.

Focus Areas:

Session Persistence

Security Concerns

Use-Case Comparison

💡 Why Session Tracking is Necessary

HTTP is a **stateless protocol**—each client request is executed independently without retaining historical context. Session tracking mechanisms allow web applications to maintain user state (e.g., authentication login status, multi-step forms, shopping cart contents) across multiple HTTP request-response cycles.

STEPS 2-4

Three Core Session Tracking Mechanisms

 **HttpSession API**

The container creates and manages a server-side session object linked to a unique session token (e.g., JSESSIONID).

```
session.setAttribute("user", "Chakri");
```

Advantages:

- Convenient high-level Java API.
- Stores complex objects server-side securely.

Limitations:

- Consumes server RAM memory.
- Requires session ID propagation mechanism.

 **Cookies**

Small text files created by the servlet, stored on the client browser, and automatically transmitted with subsequent requests.

```
Cookie c = new Cookie("theme", "dark");
response.addCookie(c);
```

Advantages:

- Simple & lightweight client-side persistence.
- Survives browser restarts (persistent cookies).

Limitations:

- Can be disabled or deleted by users.
- Vulnerable to tampering if not protected.

 **URL Rewriting**

Appends the session identifier directly to hyperlinks when client cookies are disabled or unsupported.

```
/shop;jsessionId=ABC123XYZ
```

Advantages:

- Guaranteed session tracking fallback.
- Works even when browser cookies are disabled.

Limitations:

- Session IDs leak via logs, history & referrer headers.
- Messy URL appearance.

STEP 5

Mechanism Comparison Matrix

Technique	Storage Location	Best Use Case	Main Limitation
HttpSession	Server Memory	Login states, shopping carts, multi-page workflows	Consumes server memory; requires session ID handling
Cookies	Client Browser	User preferences, themes, session tokens	Can be disabled by users; vulnerable to client tampering
URL Rewriting	URL Query Parameters	Fallback mechanism when cookies are disabled	Session tokens can leak in browser history and logs

Question-5

PAGE 5 OF 5

QUESTION 5

Cookies vs Sessions – Critical Analysis

Question: Analyze and compare Cookies and Sessions in web applications. Present real-time use cases.

Focus Areas:

Storage Location

Security Risks

Performance Considerations

STEP 1

Foundational Architectural Difference

Client-Side Cookie Architecture

Data payload resides entirely on the client browser machine. Every HTTP request sends matching cookies back to the web server via request headers.

Server-Side Session Architecture

Data payload resides securely in web server memory. Only a small, cryptographically randomly generated Session ID is shared with the client.

STEP 1 Foundational Architectural Difference

Client-Side Cookie Architecture

Data payload resides entirely on the client browser machine. Every HTTP request sends matching cookies back to the web server via request headers.

Server-Side Session Architecture

Data payload resides securely in web server memory. Only a small, cryptographically randomly generated Session ID is shared with the client.

STEP 2 In-Depth Comparison Table

Architectural Aspect	Cookies	Sessions
Storage Location	Client Browser / Local Disk	Web Server Memory / Redis Store
Typical Data Stored	User preferences, theme selections, tokens	Authenticated login state, active cart objects
Security Profile	Vulnerable to XSS / tampering if unprotected	Highly secure; state is hidden on server
Performance Impact	No server memory consumed; inflates HTTP headers	Consumes server RAM per active session
Persistence	Can persist for months (expiry dates)	Tied to browser session or server timeout
Capacity Limit	Small (~4 KB per domain limit)	High (limited only by server memory capacity)
Example Scenario	Remembering "Dark Mode" theme setting	Active user checkout & banking transactions

STEP 3 Real-Time Enterprise Use Cases



1. E-Commerce Platforms (Amazon, Flipkart)

Sessions: Hold active shopping cart items, promo codes, and logged-in user profile during checkout.
Cookies: Remember user language preferences and recently viewed item IDs across visits.



2. Online Banking & Financial Portals

Sessions: Associate authenticated requests with a strict short-timeout server session.
Cookies: Store encrypted device tokens for Multi-Factor Authentication (MFA) verification.



3. News & Media Websites

Cookies: Remember reading mode (Dark/Light), layout options, and cookie consent choices.



4. Learning Portals (SIMATS Portal)

Sessions: Maintain authenticated student state as they navigate across course modules and quizzes.

STEP 4 Industry Security Recommendations

🔒 Hardening Web Application Security

- **HTTPS Encryption:** Always deploy TLS/HTTPS to protect session tokens in transit.
- **Secure Flag:** Set Secure attribute on cookies to prevent transmission over plain HTTP.
- **HttpOnly Flag:** Set HttpOnly attribute to block client-side JavaScript access and mitigate XSS attacks.
- **SameSite Attribute:** Enforce SameSite=Strict or Lax to prevent Cross-Site Request Forgery (CSRF).
- **Session Fixation Prevention:** Regenerate session IDs immediately after successful user authentication.
- **Explicit Invalidation:** Always invoke session.invalidate() upon user logout.

STEP 5 Final Architectural Conclusion



Summary Conclusion

Cookies and Sessions are **complementary technologies** rather than strict competitors. Modern web applications utilize a hybrid approach: storing a cryptographically secure session identifier in an HttpOnly, Secure cookie on the client while managing sensitive state objects securely on the web server.